
Universidad Tecnológica Metropolitana.
Departamento de Computación e Informática.
Computación Paralela y Distribuida
Profesor: Sebastián Salazar Molina.

Servicio ReST:

Resumen estadístico.

Entrega 17 de Julio de 2026.

Contexto.

Cruz Morada, una de las cadenas de farmacias líderes en el mercado chileno, requiere implementar un servicio REST que permita obtener un resumen estadístico integral de sus datos de ventas. Históricamente, la información de transacciones y clientes se almacenaba de manera fragmentada, lo que dificultaba el análisis estratégico. Ahora, con un archivo CSV consolidado que centraliza toda la información relevante, se busca automatizar la generación de métricas clave mediante una API eficiente.

El servicio debe procesar este archivo de gran volumen, aplicando técnicas de procesamiento paralelo y distribuido, para garantizar un rendimiento óptimo y escalable. El objetivo es entregar insights accionables, como: suma, conteo de filas, promedio, mínimo, máximo, mediana y desviación estándar. Para identificación de patrones y tendencias comerciales.

Esta solución eliminará la necesidad de procesos manuales de integración y limpieza, permitiendo a los analistas enfocarse en el análisis avanzado y la toma de decisiones basadas en datos.

Objetivos de Aprendizaje

Objetivo principal.

Diseñar e implementar una solución computacional para Cruz Morada que permita procesar y analizar sus datos de ventas mediante un servicio REST, el cual deberá:

- Procesar archivos CSV de gran volumen de manera eficiente.
- Implementar mecanismos de procesamiento paralelo para garantizar escalabilidad y bajo consumo de recursos.
- Incluir documentación técnica en formato Swagger o ReadTheDocs para facilitar su uso e integración.
- Cargar los datos de manera desatendida (ej.: al iniciar la aplicación mediante una operación CLI o un script de arranque).

Objetivos Específicos

Al finalizar este trabajo práctico, los estudiantes serán capaces de:

1. Diseñar e implementar un servicio REST con dos operaciones principales:
 - a. GET: Consultar totales y métricas precomputadas.
 - b. POST: Realizar consultas dinámicas con parámetros.
2. Implementar un mecanismo de carga desatendida:
 - a. Procesar y almacenar el archivo CSV al iniciar la aplicación (vía CLI, script o servicio en segundo plano).
 - b. Garantizar que los datos estén disponibles para las consultas GET y POST sin intervención manual.
3. Modelar estructuras de datos eficientes:
 - a. Optimizar el almacenamiento temporal o en memoria para un acceso rápido a los datos procesados.
 - b. Opcional: Usar una (o varias) bases de datos, pueden ser relacionales como NoSQL.
4. Implementar procesamiento paralelo:
 - a. Utilizar técnicas como chunking, streaming o librerías (ej.: Dask, PySpark) para manejar grandes volúmenes de datos.
5. Documentar la solución:
 - a. Generar documentación en Swagger o ReadTheDocs que incluya:
 - i. Especificaciones detalladas de los endpoints (GET y POST).
 - ii. Ejemplos de solicitudes, respuestas y parámetros.

Lenguaje de Programación y Frameworks

- **Lenguaje de programación:** Los estudiantes pueden elegir el lenguaje que mejor se adapte a sus necesidades, siempre que dicho lenguaje sea OpenSource y se pueda instalar de forma nativa en GNU/Linux.
- **Frameworks y bibliotecas:** Se recomienda el uso de herramientas open source para el desarrollo del proyecto
 - Python: FastAPI, Flask o Bottle (entre otros).
 - Java: SpringBoot, Quarkus o Micronaut (entre otros).
 - NodeJS: Express.js, Fastify, NestJS (entre otros).
 - Otros: Cualquier framework que permita la generación de servicios ReST.

Especificaciones Técnicas

Datos de Entrada

- Archivo CSV compartido por carpeta drive:
 - <https://drive.google.com/file/d/15jLBIJ9eMQSoHsoCMnFWBGopr98FIHIK/view?usp=sharing>
- Cada grupo debe proveer algún mecanismo para cargar este archivo en el proyecto.

Estructura de los Archivos CSV

Cada archivo contiene las siguientes columnas (separadas por comas):

Columna	Tipo	Descripción	Ejemplo
FECHA	String	Fecha de la operación en formato ISO 8601 (UTC-4).	2026-05-08T00:02:53
CANAL	String	Canal de compra (ej: POS, online).	POS
SKU	Integer	Identificador único del producto.	1095
PRODUCTO	String	Nombre del producto.	EUCERIN SERUM DERMOP.40ML
UNIDADES	Integer	Cantidad de unidades compradas.	1
PORCENTAJE DESCUENTO	Float	Porcentaje de descuento aplicado (0 a 1).	0.15
MONTO APLICADO	Float	Monto total pagado en CLP (pesos chilenos).	12500.0
BOLETA	Integer	Número de boleta de la transacción.	100456

Columna	Tipo	Descripción	Ejemplo
LOCAL	Integer	Identificador del local donde se realizó la compra.	1999
CODIGO CLIENTE	String (UUID)	Identificador único del cliente (formato UUID v3).	550e8400-e29b-41d4-a716-446655440000
RUN CLIENTE	String	RUT del cliente (formato chileno).	12.345.678-5
NOMBRES	String	Nombre del cliente.	JUAN
APELLIDOS	String	Apellido(s) del cliente.	PÉREZ GÓMEZ
FECHA NACIMIENTO	String	Fecha de nacimiento del cliente (formato AAAA-MM-DD).	1995-08-15
GÉNERO	Integer	Identificador del género: 1. Masculino. 2. Femenino.	1

TRABAJO.

Requisitos Funcionales

Endpoint Base

Se deberá levantar un servidor web que sirva las operaciones desde la siguiente ruta base: **/v1/estadisticas/ventas**

Esta operación se deberá consultar usando los métodos soportados:

- **GET.** Devuelve estadísticas de ventas con filtros predeterminados (vía query params opcionales).
- **POST.** Devuelve estadísticas de ventas con filtros personalizados (opcionales).

Estructuras de Datos

Respuesta Exitosa (JSON)

```
{
  "suma": 1500.5,
  "conteo": 42,
  "promedio": 35.73,
  "minimo": 10.0,
  "maximo": 100.0,
  "mediana": 30.0,
  "desviacion_estandar": 25.4
}
```

Solicitud POST (Body JSON)

```
{
  "consultas": [
    {"consulta": "GENERO", "valor": "Femenino"},
    {"consulta": "EDAD", "valor": "31"},
    {"consulta": "CANAL", "valor": "POS"}
  ]
}
```

Filtros soportados deben ser textualmente uno o varios de estos:

- **GENERO.** Los valores permitidos son:
 - "No especificado".
 - "Masculino".
 - "Femenino".
 - "Otro".
- **EDAD.** Número entero que representa la edad a consultar.
- **CANAL.** Indica el medio mediante el cuál se realizó la venta, los valores posibles pueden ser:
 - POS
 - WEB
 - APP
 - CCT
 - APR
 - WPR
- **CODIGO_PRODUCTO.** Es el identificador único de la persona.
- **ID_PERSONA.** Es el código UUID que identifica un cliente.
- **LOCAL.** Es el número de local.
- **FECHA_DESDE.** La fecha desde donde se buscarán las compras, en formato ISO-8601.
- **FECHA_HASTA.** La fecha hasta donde se buscarán las compras, en formato ISO-8601.

Las consultas pueden realizarse sin usar estos filtros o usando cualquier cantidad arbitraria de los mismos.

Formato de Errores

Todas las respuestas de error **deben** seguir este formato exacto:

```
{
  "detail": "Descripción detallada del error (ej: 'El valor X no es válido para Y')",
  "instance": "/v1/estadisticas/ventas",
  "status": 400,
  "title": "Bad Request",
  "type": "https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status/400",
  "timestamp": "2026-06-30T20:44:49.201437123Z",
  "errorCode": "VF",
  "errorLabel": "Validación Fallida",
  "method": "POST"
}
```

Requisitos Técnicos

Validaciones y Errores

Código 400 Bad Request (Validación fallida)

- consultas vacío o nulo.
- consulta no es uno de los valores permitidos.
- valor no es convertible al tipo esperado.

Ejemplo:

```
{
  "detail": "El valor 'qwerqwer' no es un número entero válido para el ID de tienda",
  "instance": "/v1/estadisticas/ventas",
  "status": 400,
  "title": "Bad Request",
  "type": "https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status/400",
  "timestamp": "2026-06-30T20:44:49.201437123Z",
  "errorCode": "VF",
  "errorLabel": "Validación Fallida",
  "method": "POST"
}
```

Código 500 Internal Server Error

Ejemplo:

```
{
  "detail": "Error al calcular la desviación estándar",
  "instance": "/v1/estadisticas/ventas",
  "status": 500,
  "title": "Internal Server Error",
  "type": "https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status/500",
  "timestamp": "2026-06-30T20:44:49.201437123Z",
  "errorCode": "IE",
  "errorLabel": "Error Interno",
  "method": "GET"
}
```

Cálculo de Estadísticas

Fórmulas:

- promedio = suma / conteo
- mediana: Valor central (si conteo es par, promedio de los 2 centrales).
- desviacion_estandar: Raíz cuadrada de la varianza.

Entrega

1. Código fuente de la API.
2. Archivo README.md con instrucciones y ejemplos.
3. Archivo datos.json con datos de prueba.
4. Pruebas unitarias (opcional).

Entregables

La fecha de entrega es el 17/07/2026 hasta las 23:59:59.999999 hora continental de Chile. Los estudiantes deben entregar:

- 1. Código fuente de la API en repositorio GitHub, deben incluir al académico como colaborador (sebasalazar).
- 2. Archivo README.md con instrucciones y ejemplos.
- 3. Archivo datos.json con datos de prueba.
- 4. Pruebas unitarias.

Rúbrica de Evaluación

Criterio	Peso
Funcionalidad de endpoints	30%
Formato de errores	20%
Cálculo correcto de estadísticas	20%
Manejo de filtros y validaciones	15%
Código limpio y documentado	15%